

Space Debris Detection & Collision Avoidance

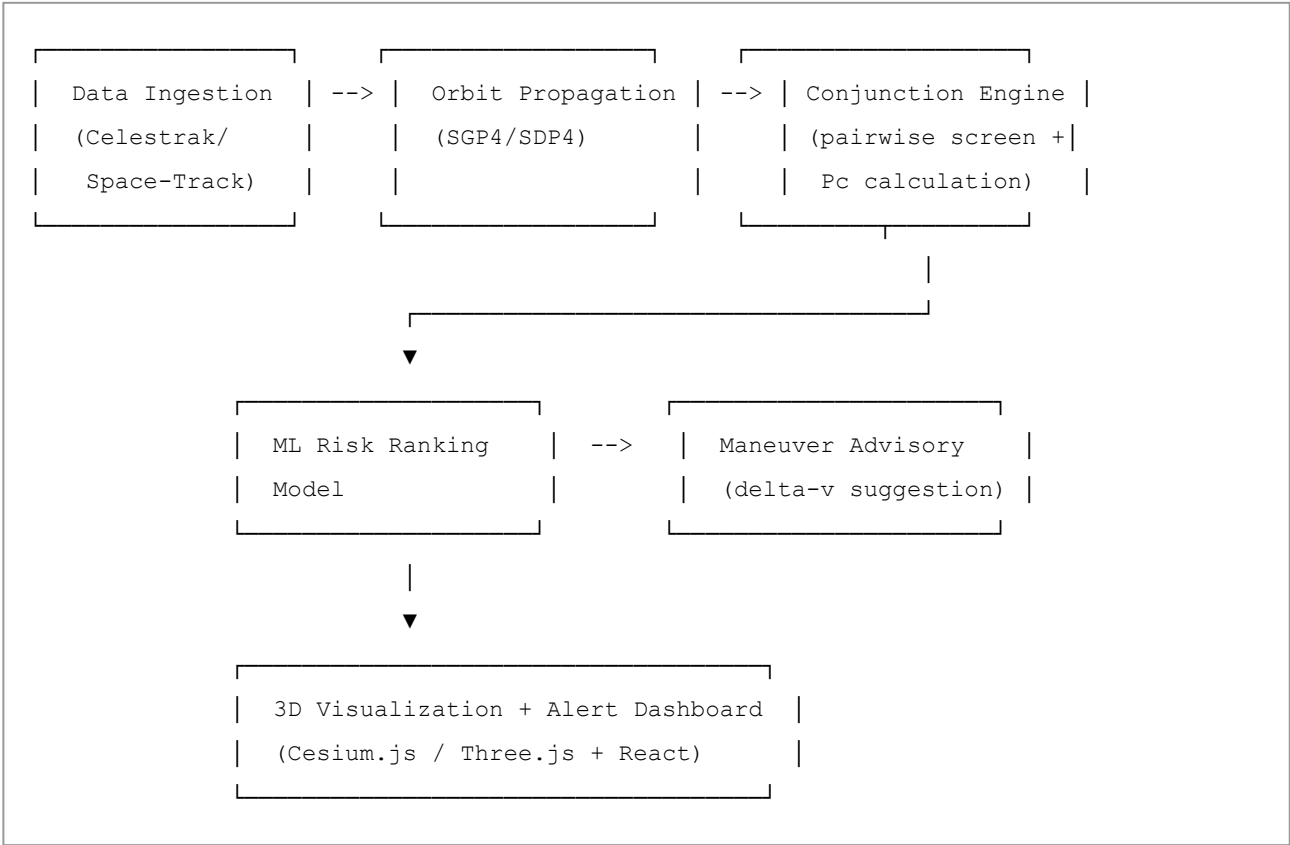
Full Hackathon Blueprint

1. Project Summary

One-line pitch: A real-time orbital collision risk platform that ingests live satellite tracking data, propagates orbits using real aerospace algorithms (SGP4), computes collision probabilities using the same math used by actual space agencies, and uses ML to prioritize which conjunctions need human attention — visualized on a live 3D globe.

Why this avoids "AI slop" criticism: The core engineering is physics and algorithms (orbit propagation, geometry, probability theory). ML is used as a *decision-support layer* on top of real computation, not as a replacement for it. Judges can inspect the propagation math, the conjunction geometry, and the Pc (probability of collision) calculation independently of the ML — that's what makes it defensible.

2. System Architecture



3. Tech Stack

Layer	Tech Choice	Notes
Data ingestion	Python, <code>requests</code> , Celestrak API	Free, no auth needed for basic TLE data
Orbit propagation	<code>sgp4</code> Python library (or <code>python-skyfield</code>)	Industry-standard implementation, don't write SGP4 from scratch (too risky in 2 weeks)
Numerical/geometry	<code>numpy</code> , <code>astropy</code>	Coordinate frame transforms (TEME → ECI → ECEF)
Conjunction screening	Custom Python (k-d tree via <code>scipy.spatial</code> for pairwise proximity)	Avoids $O(n^2)$ brute force on thousands of objects
Pc calculation	Custom implementation of Foster's 2D method, or Monte Carlo fallback	This is your "we did real aerospace math" flex
ML risk model	<code>scikit-learn</code> (Gradient Boosting) or <code>XGBoost</code>	Interpretable, fast to train, easy to explain to judges
Backend API	FastAPI (Python)	Clean, async, auto-generates docs (judges like seeing <code>/docs</code>)
Database	PostgreSQL + PostGIS, or simple SQLite for hackathon scope	Store TLEs, conjunction events, historical risk scores
Frontend	React + Cesium.js (best for real orbital globes) or Three.js (lighter weight)	Cesium has built-in satellite/orbit rendering support
Deployment	Docker Compose, deploy on Render/Railway/local for demo	Keep it simple — don't over-engineer deployment

4. Data Sources (all free, no special access needed)

1. **Celestrak** (celestrak.org) — live TLE data for thousands of tracked objects, updated regularly, no auth required
 2. **Space-Track.org** — more comprehensive catalog (requires free account), includes conjunction data messages (CDMs) for reference/validation
 3. **NASA ODPO** (Orbital Debris Program Office) — public debris environment reports, good for your problem framing slides
 4. **ESA Space Debris Office** — annual environment reports, useful citations for your pitch deck
-

5. Core Algorithms (the "not slop" backbone)

5.1 Orbit Propagation (SGP4)

- Every tracked object has a TLE (two-line element set) describing its orbit.
- SGP4 propagates that TLE forward in time to predict position/velocity at any future timestamp.
- Use the `sgp4` Python package — implementing this from scratch is not a good use of 2 weeks, but you must understand and be able to explain it (judges will ask).

5.2 Conjunction Screening

- With thousands of objects, checking every pair at every timestep is $O(n^2)$ — too slow.
- Use a coarse filter first (e.g., only compare objects with similar orbital altitude bands), then fine-screen with a k-d tree or simple distance threshold (e.g., objects predicted within 10 km of each other).
- This two-stage filter (coarse + fine) is a real systems-engineering decision you can explain and defend.

5.3 Probability of Collision (P_c)

- Given a predicted close approach, position uncertainty (covariance) determines actual collision risk — a "close" approach with high uncertainty is scarier than a "closer" approach with well-known positions.
- Implement **Foster's 2D P_c method**: projects the relative position and combined covariance onto a 2D plane perpendicular to the relative velocity vector, then integrates a Gaussian probability density over a circle representing combined object size.
- If time is short, a Monte Carlo fallback (sample many possible positions from the uncertainty distribution, count how many result in collision) is easier to implement and still legitimate.

5.4 ML Risk Ranking (this is where AI earns its place)

- **Problem framing**: with hundreds/thousands of daily conjunctions flagged by the geometric screen, most are false alarms. ML prioritizes which ones a human operator should actually review.
- **Features**: miss distance, relative velocity, combined object cross-sectional area, orbital regime (LEO/MEO/GEO), historical P_c trend, object type (active satellite vs debris).
- **Model**: Gradient Boosted Trees (XGBoost/LightGBM) — trains fast, interpretable via SHAP values (great for a judge Q&A: "why did the model flag this one?").
- **Labels**: if you don't have real incident labels, use P_c threshold from your physics engine as a weak label, or simulate synthetic high-risk scenarios to validate the model actually learns something beyond just repeating P_c .
- **Honesty point for judges**: be upfront that the ML augments a real physics computation rather than replacing it. This is the single strongest thing you can say to preempt "AI slop" criticism.

5.5 Maneuver Advisory

- For flagged high-risk conjunctions, compute a simple delta-v suggestion (e.g., a small radial or along-track burn) that would increase miss distance beyond a safety threshold.
- Doesn't need to be operationally perfect — showing you understand the *type* of maneuver (and the tradeoff between fuel cost and risk reduction) is what matters for the demo.

6. Team Split (4 people)

Role	Owns	Key deliverable
Orbital Mechanics Lead	TLE ingestion + SGP4 propagation + coordinate transforms	Working propagation pipeline, validated against known satellite positions
Conjunction/Math Lead	Screening algorithm + Pc calculation	Conjunction detection engine with probability outputs
ML Lead	Feature engineering + risk ranking model + explainability	Trained model + evaluation metrics + SHAP plots
Frontend/Viz Lead	Cesium.js 3D globe + dashboard + alert UI	Live visualization, this is your demo centerpiece

Backend/API work can be split across all 4 as a shared thin layer connecting the pieces.

7. Two-Week Timeline

Week 1 — Core Engine

- Days 1-2: Set up TLE ingestion, get SGP4 propagation working and validated (compare against known satellite pass predictions)
- Days 3-4: Build conjunction screening (coarse + fine filter), get basic Pc calculation working on a small object set
- Days 5-7: Expand to full catalog scale, start feature extraction for ML, begin Cesium globe skeleton (render live TLE positions)

Week 2 — Intelligence + Polish

- Days 8-9: Train and validate ML risk ranking model, integrate into pipeline
- Days 10-11: Build maneuver advisory logic, connect frontend to backend (live alerts on globe)
- Days 12-13: Polish dashboard (analytics panel, risk timeline, SHAP explainability view), rehearse demo flow
- Day 14: Buffer day — fix bugs, prepare pitch deck, record backup demo video (in case live demo fails on stage)

8. Demo Script (3-minute pitch structure)

1. **Problem** (20 sec): Show real stat — 130M+ pieces of debris tracked/estimated, Kessler syndrome risk, real agencies spend enormous effort on this manually.
2. **Live globe** (60 sec): Show the 3D visualization with live-ish TLE data, zoom into a flagged conjunction, show the two objects and their predicted close approach.

3. **Under the hood** (60 sec): Briefly show the Pc calculation output and the ML risk ranking with SHAP explanation — this is where you kill the "slop" accusation by showing real math and real model reasoning side by side.
 4. **Maneuver advisory** (20 sec): Show the suggested avoidance maneuver and the resulting improved miss distance.
 5. **Close** (20 sec): Impact statement — this kind of triage tool reduces analyst workload from reviewing thousands of alerts to reviewing dozens.
-

9. Risk Mitigation (things that commonly break in 2 weeks)

- **SGP4 accuracy confusion:** TLEs degrade in accuracy over time — don't propagate more than a few days forward or your positions will visibly drift. Use fresh TLEs.
 - **Scale issues:** Don't try to process the entire 20,000+ object catalog live in the browser. Pre-filter to a demo-relevant subset (e.g., 500-1000 objects in LEO) for smooth real-time rendering.
 - **Coordinate frame bugs:** The #1 source of "satellites are floating in the wrong place" bugs is mixing up TEME/ECI/ECEF frames. Budget real time for this — use `astropy` or `skyfield` to handle transforms rather than writing your own.
 - **Have a recorded backup demo.** Live 3D rendering + live data during a hackathon Wi-Fi environment is inherently risky.
-

10. Hardware Extension (optional, if you want a physical component)

You don't *need* hardware for this to be impressive — the software depth already carries it. But if you want a physical/hardware layer to make the demo even more tangible, here are realistic options for a 2-week student build:

Option A: RTL-SDR Ground Station (most impressive, moderate cost/effort)

- Use a cheap RTL-SDR USB dongle (~\$2,000-3,000) with an antenna to **actually receive real satellite signals** (e.g., NOAA weather satellites broadcast on 137 MHz, or amateur radio satellite beacons).
- Decode real telemetry or weather images live on stage. This is a genuine "we built a real ground station" moment that almost no other team will have.
- Libraries: `rtl-sdr`, `gr-satellites` (GNU Radio satellite decoding), `noaa-apt` for weather image decoding.
- This ties directly into your theme: your software tracks orbits, your hardware physically listens to a real satellite pass overhead during the demo.

Option B: Physical Orbit Visualization Model

- A small motorized globe or LED-ring display (Arduino/Raspberry Pi driven) that physically lights up LEDs representing tracked objects' current positions, synced with your software's live data via serial/WiFi.
- Less technically deep than Option A, but a strong visual/tactile demo piece for a table booth.

Option C: GPS-based Conjunction Simulation Rig

- Use 2 small ESP32 boards with GPS modules or IMU-based dead reckoning to simulate "two objects on a collision course," streaming position data into your Pc calculation engine live — a physical stand-in for two satellites, letting you demo the collision-avoidance math on real (if simplified) moving hardware rather than only simulated data.

Recommendation

If your team has any hardware/embedded skill among the 4 people, **Option A (RTL-SDR ground station)** is the strongest choice — it's relatively cheap, well-documented, genuinely impressive ("we're receiving a real satellite signal right now"), and directly reinforces your project's credibility as more than "just an ML wrapper." Budget 2-3 days for it as a parallel track, not on the critical path — if it doesn't work perfectly, your core software demo still stands on its own.

11. Key Libraries/Repos to Start From

- `sgp4` (Python) — <https://pypi.org/project/sgp4/> (<https://pypi.org/project/sgp4/>).
- `skyfield` — higher-level astronomy library, good for coordinate transforms and visibility calculations
- `astropy` — coordinate frames, time handling
- CesiumJS — <https://cesium.com/platform/cesiumjs/> (<https://cesium.com/platform/cesiumjs/>) (has satellite/orbit examples in their sandcastle demos)
- `gr-satellites` — for the RTL-SDR hardware extension, if pursued